

# AETHERIA — Database Design

Step 2 · logical model · target: PostgreSQL 16 · DB name: aetheria

**Conventions** - Primary keys = `bigserial` unless noted. - All timestamps = `timestamptz`, default `now()`. - Soft delete = `deleted_at timestamptz NULL`. - JSONB for flexible / data-driven payloads. - Indexes documented inline; FKs ON DELETE chosen per relation semantics.

## 1. Entity Map (textual ERD)

```
users ■■■■< profiles
    ■■< user_characters >■■ characters
    ■■■< character_skills >■■ skills
    ■■< user_skills >■■ skills
    ■■< inventory >■■ items
    ■■< runs >■■ levels >■■ realms
    ■■< save_states
    ■■< user_quests >■■ quests
    ■■< guild_members >■■ guilds
    ■■< pvp_match_players >■■ pvp_matches
    ■■< friendships
    ■■< chat_messages
    ■■< battle_pass_progress >■■ battle_pass_seasons
    ■■< audit_log
leaderboards (materialized + Redis ZSET mirror)
```

## 2. Tables

### 2.1 Identity & Profile

users

| col                                   | type        | constraints               | note                         |
|---------------------------------------|-------------|---------------------------|------------------------------|
| id                                    | bigserial   | PK                        |                              |
| email                                 | citext      | UNIQUE NOT NULL           |                              |
| password_hash                         | text        | NULL                      | argon2id; NULL if OAuth-only |
| oauth_provider                        | text        | NULL                      | google/discord               |
| oauth_subject                         | text        | NULL                      |                              |
| email_verified_at                     | timestamptz | NULL                      |                              |
| status                                | text        | NOT NULL DEFAULT 'active' | active/banned/suspended      |
| created_at, updated_at, last_login_at | timestamptz |                           |                              |
| deleted_at                            | timestamptz | NULL                      |                              |

Indexes: (`oauth_provider`, `oauth_subject`), `lower(email)`.

profiles — 1:1 with users

- `user_id` PK FK → `users.id`

- display\_name (citext UNIQUE)
- avatar\_url, country (iso2), language (bcp47)
- account\_level int default 1, account\_xp int default 0
- preferences jsonb default '{}'

## 2.2 Characters & Skills (master + ownership)

characters (catalog of 8 starters, expandable)

- id, codename UNIQUE, name, class, role, base\_stats jsonb, art\_url, lore text, unlock\_requirement jsonb, version int

skills

- id, character\_id FK → characters.id, name, type (active/passive), ap\_cost int, cooldown int, effect jsonb, icon\_url

character\_skills — pivot of skill order per character

- character\_id, skill\_id, slot int (1..6) — PK(character\_id, slot)

user\_characters — user owns/levels chars

- id, user\_id FK, character\_id FK, ascension int default 0, xp int, equipped\_skin\_id FK NULL, unlocked\_at
- UNIQUE(user\_id, character\_id)

user\_skills — per-user skill investment

- user\_id, skill\_id, level int (1..10), unlocked\_at — PK composite

## 2.3 World Content

realms — id, name, theme, color\_hex, order\_index levels

- id, realm\_id FK, level\_number int (1..100 globally), name, type (story/combat/puzzle/treasure/boss/hidden/rift)
- map jsonb (hex grid)
- encounter jsonb (waves, enemies, AI)
- rewards jsonb
- difficulty int
- min\_account\_level int
- discovery\_secrets jsonb
- version int
- UNIQUE(level\_number)

## 2.4 Items & Inventory

items — id, name, tier (common/rare/epic/mythic/aetherforged), type (weapon/armor/relic/consumable/material), effect jsonb, icon\_url, max\_stack int inventory

- id, user\_id, item\_id, quantity int, equipped\_to\_user\_character\_id NULL

- UNIQUE(user\_id, item\_id) (when stackable)

## 2.5 Runs / Save States

runs — a playthrough attempt of a level

- id, user\_id, level\_id, status (in\_progress/completed/failed/abandoned)
- started\_at, ended\_at, score int, stars int (0..3)
- action\_log jsonb (compact log for replay/anti-cheat)
- snapshot jsonb (final mid-state for resume)

save\_states

- id, user\_id, slot int (0..3), payload jsonb, schema\_version int, updated\_at
- UNIQUE(user\_id, slot)

## 2.6 Quests / Battle Pass

quests — id, type (daily/weekly/seasonal/story), requirements jsonb, rewards jsonb, active\_from, active\_to

user\_requests — user\_id, quest\_id, progress jsonb, status (active/completed/claimed), claimed\_at — PK

composite battle\_pass\_seasons — id, name, starts\_at, ends\_at, tracks jsonb (free/premium per tier)

battle\_pass\_progress — user\_id, season\_id, tier int, premium boolean, xp int — PK composite

## 2.7 Social

guilds — id, name UNIQUE, tag (4 chars) UNIQUE, description, leader\_user\_id FK, level int, xp int,

created\_at guild\_members — guild\_id, user\_id, role (leader/officer/member), joined\_at, contribution int —

PK composite friendships — user\_id, friend\_id, status (pending/accepted/blocked), created\_at —

PK(user\_id, friend\_id), friend\_id < user\_id rule via trigger or always store both chat\_messages

- id, channel\_type (global/guild/party/whisper), channel\_id (nullable for global), sender\_id, content text, created\_at, flagged boolean default false
- BRIN index on created\_at; partition by month

## 2.8 PvP & Ranking

pvp\_matches

- id, mode (1v1/3v3/raid), status, started\_at, ended\_at, winner\_user\_id NULL, region, server\_seed bytea

pvp\_match\_players

- id, match\_id FK, user\_id FK, lineup jsonb (3 user\_character\_ids + items), score int, mmr\_before int, mmr\_after int, result (win/loss/draw)

mmr — user\_id PK, mode PK, mmr int, peak\_mmr int, season\_games int, season\_wins int leaderboards

— materialized table; primary mirror is Redis ZSET key lb: {mode}: {season\_id} for live updates; nightly snapshot to this table.

## 2.9 Marketplace (cosmetic / soft-currency only)

shop\_items — id, item\_id FK, currency\_type (gold/aether), price int, available\_from, available\_to, stock int

NULL transactions — id, user\_id, shop\_item\_id, currency\_type, amount int, status, created\_at

## 2.10 System

`audit_log` — `id`, `actor_user_id` NULL, `action` text, `target_type`, `target_id`, `payload` jsonb, `ip` inet, `user_agent` text, `created_at`; partitioned monthly  
`feature_flags` — `key` text PK, `value` jsonb, `updated_by`, `updated_at`  
`migrations` — handled by Prisma

### 3. Indexing Strategy

- Hot read keys: `inventory(user_id)`, `user_requests(user_id, status)`, `runs(user_id, status)`.
- Leaderboard reads happen via Redis (ZRANGE), so no heavy SQL index on `mmr`.
- `chat_messages` partitioned monthly + BRIN on `created_at`.
- `audit_log` partitioned monthly.

### 4. Cardinality / Sizing Sketch (10 k DAU baseline)

- users / profiles: 100 k rows over year 1
- runs: 30 / DAU / day → ~9 M rows / month → partition `runs` monthly after MVP.
- chat\_messages: 200 / DAU / day → ~60 M / month → partition mandatory.
- audit\_log: ~5 M / month → partition.
- inventory: ~30 rows / user.

### 5. Data Retention

- chat\_messages: 90 days then archive to R2 (parquet).
- audit\_log: 1 year hot, then cold archive.
- runs: keep snapshots forever for top-100 replays; trim others to 30 days.
- pvp\_matches: 1 season hot, archive after.

### 6. JSONB Schemas (high level — full schemas live in packages/schema-db/json)

- `levels.map` → { width, height, tiles:[{q,r,terrain,elev,fx}], spawns, exits }
- `levels.encounter` → { waves:[{enemies:[...] }], boss?, scripts? }
- `levels.rewards` → { xp, gold, items:[{itemId, qty}], firstClearBonus }
- `runs.action_log` → array of { t, actor, type, data } events
- `save_states.payload` → entire scene state + party + inventory delta

### 7. Naming

- Tables snake\_case plural; PK `id`; FK `<entity>_id`.
- DB name: `aetheria`. Roles: `aetheria_app`, `aetheria_readonly`, `aetheria_migrator`.